

DashBoard Open Source

📅 Date	July 16, 2026
🔗 URL	https://github.com/daxAKAhackerman/suricata-lightweight-gui

Dashboard Setup, Debugging & Custom Code Modification

Choosing a Dashboard

Searched for existing Python-based, open-source dashboards for visualizing `eve.json`. Ruled out EveBox (Go-based, not Python) and settled on `suricata-lightweight-gui` (GitHub: daxAKAhackerman) — a Flask (Python) backend + VueJS frontend, purpose-built to bind directly to `eve.json`.

Installing Docker (required by the project's setup method)

bash

```
sudo apt install docker.io docker-compose -y sudo systemctl enable --now docker
```

Why: the project's documented install method is `docker-compose up -d`, so Docker itself had to be installed first. Discovered the system actually installed the newer `docker-compose-v2` plugin, requiring the modern `docker compose` (two words) syntax instead of the legacy hyphenated `docker-compose` command.

Debugging the Build (multiple real compatibility issues, ~5-year-old project)

Issue 1 — Python version incompatibility

Original build failed because `Dockerfile.backend` used `FROM python:latest`, which pulled Python 3.14 — too new for the project's 2020-era pinned dependency (`autopep8==1.5.2`), causing a build error.

Fix: removed `autopep8` and `pycodestyle` from `requirements.txt` (confirmed these were dev-only linting tools, not runtime dependencies) and pinned the Dockerfile to

`FROM python:3.9` — an older, stable version matching the project's actual era.

Issue 2 — Node.js version too old

`Dockerfile.frontend` tried installing Node.js 10.x via an outdated NodeSource setup script, which failed because the current Ubuntu release ("resolute") isn't recognized by that deprecated script.

Fix: changed the script URL from `setup_10.x` to `setup_18.x`, installing a modern, supported Node LTS version instead.

Issue 3 — `npm audit fix` failing on its own

The Dockerfile ran `npm audit fix`, which exited with an error because many of the 112 flagged vulnerabilities in this old dependency tree required `--force` (breaking changes) to resolve, and the script wasn't passing that flag.

Fix: removed the `RUN npm audit fix` line entirely — confirmed it wasn't required for the app to actually build or function, just an optional security-patching step.

Issue 4 — OpenSSL legacy hashing algorithm error

`npm run build` failed with `error:0308010C:digital envelope routines::unsupported` — a well-documented, common issue where old Webpack versions rely on an MD4 hash function that Node 18's newer OpenSSL 3 no longer supports by default.

Fix: changed the build command to `NODE_OPTIONS=--openssl-legacy-provider npm run build`, restoring the legacy cryptographic behavior needed by the old build tooling.

Result: after these four fixes, `sudo docker compose up -d --build` completed successfully, with both `backend-1` and `frontend-1` containers running.

Accessing and Using the Dashboard

Verified via browser at `http://192.168.93.140:8888` — dashboard correctly displayed real alert data from earlier nmap/nikto testing, with working sort, filter (including regex support), and pagination.

Note on case sensitivity: discovered filtering "nmap" (lowercase) returned zero results because the actual signature text is "Nmap" (capital N) and the "Make search case insensitive" checkbox was unchecked — not a bug, just a usage detail.

Custom Code Modification — Live Auto-Refresh

Problem identified: dashboard only fetched data once on page load (`created() { this.getData(); }`), requiring a manual "Fetch" button click to see new alerts — not truly "live."

Located the relevant file:

bash

```
find. -type f -name "*.vue"
```

found `App.vue` and `client/src/components/RenderJSON.vue` — the latter contains the actual data-fetching logic.

Code changes made (in `RenderJSON.vue`):

1. Added `pollInterval: null` to the component's `data()` state, to store a reference to the timer
2. Added to `mounted()` lifecycle hook:

javascript

```
this.pollInterval=setInterval(()=>{this.getData();},1000);
```

Why: `setInterval` repeatedly calls `getData()` automatically every 1000ms (1 second), rather than waiting for a manual click

3. Added a `beforeDestroy()` lifecycle hook:

javascript

```
beforeDestroy(){clearInterval(this.pollInterval);}
```

Why: standard cleanup practice — stops the timer if the component is ever removed, preventing a memory leak / orphaned timer

4. Updated `getData()` to also refresh `totalRows` on every fetch (previously only set once, in `mounted()`), keeping pagination counts accurate as new data streams in continuously

Rebuild and fix during this stage: hit an ESLint error (`Mixed spaces and tabs`) caused by an accidental Tab character introduced while editing in `nano` — fixed by locating the exact line and retyping indentation using only spaces, confirmed via `grep -nP '\t' <file>` to detect literal tab characters.

Verification: confirmed the dashboard now updates automatically without manual refresh, by running live attack traffic from Kali and watching new rows appear within 1 second, unprompted.

Continuous Test Traffic Generation

To populate the live-updating dashboard with a steady stream of varied data for demonstration:

bash

```
while true; do curl -s http://192.168.93.139> /dev/nullsleep1 nmap -sV192.168.93.139> /dev/null2>&lsleep1curl -s"http://192.168.93.139/?id=1' OR '1'='1"> /dev/nullsleep1 gobusterdir -u http://192.168.93.139 -w /usr/share/wordlists/dirb/common.txt -q> /dev/null2>&lsleep1done
```

Why: cycles through benign traffic (curl), reconnaissance (nmap), an injection-style probe, and directory brute-forcing (gobuster), each separated by short pauses — gives the live dashboard a continuously varied, realistic mix of alert and non-alert traffic to display.

Observation noted: at very tight timing (1-second polling combined with rapid, overlapping loop requests), Suricata generated a large volume of low-priority "SURICATA HTTP unable to match response to request" noise, which could crowd out the more meaningful signature-based alerts in the dashboard's fixed-size fetch window — addressed by using the Signature filter to isolate genuine alerts from background noise, and by slightly increasing sleep intervals between loop steps.